# Car.Ma._Cell

Paired analysis of mitochondrial networks
in fluorescence micrographs

## User Manual

Version 1.1.0 • November 2026

Carlo Olivieri  Mariarosaria Negri  Salvatore Sessa

Source code and releases: https://github.com/colivieri89/Car.Ma._Cell

Released under the MIT License.

# Contents


**1 Introduction** 3

**2 Installation** 4
- 2.1 Getting the program . . . 4
- 2.2 Windows . . . 4
- 2.3 macOS . . . 4
- 2.4 Linux and other systems . . . 5
- 2.5 Updating and removing . . . 5

**3 Quick start** 6

**4 The program window** 8
- 4.1 Toolbar . . . 8
- 4.2 Tabs . . . 8
- 4.3 Zoom and pan . . . 11
- 4.4 Excluding areas . . . 12
- 4.5 Settings . . . 12
- 4.6 Appearance . . . 12

**5 Preparing the images** 13

**6 How the analysis works** 14
- 6.1 Calibration and common grid . . . 14
- 6.2 The pipeline . . . 14
- 6.3 Advanced parameters . . . 16

**7 Metrics** 17
- 7.1 Topology and lengths . . . 17
- 7.2 Cell normalisation . . . 17
- 7.3 Branching . . . 17
- 7.4 Connectivity and fragmentation . . . 17
- 7.5 Network complexity index . . . 18
- 7.6 Object shape and fractal dimension . . . 18
- 7.7 Node-degree distribution . . . 18
- 7.8 Distribution statistics . . . 18
- 7.9 The bar chart . . . 18
- 7.10 Example . . . 19

**8 Output files** 20
- 8.1 Excel workbook . . . 20
- 8.2 Example results . . . 20

**9 Command line and Python** 21
- 9.1 Paired analysis from the command line . . . 21
- 9.2 In a Python script . . . 21
- 9.3 Single images: the package . . . 21
- 9.4 Tests . . . 21

**10 Troubleshooting** 22

**11 Citation, licence and authors** 23

# 1 Introduction

Car.Ma._Cell compares the mitochondrial networks of two cells imaged by fluorescence microscopy. The two images, labelled A and B (by default *Fusion* and *Fission*), go through the same pipeline with the same parameters, so that a difference between the two sets of results reflects the cells and not the settings. For each cell the program extracts the skeleton of the mitochondrial network, turns it into a graph of branches, junctions and endpoints, and reports more than one hundred and fifty quantities side by side: topology, branch lengths, density, branching, connectivity, fragmentation, network complexity, object shape, fractal dimension and node-degree distribution.

The analysis runs in five steps, each shown in its own tab of the program window:

1. Inputs. The two images are read, reduced to one intensity channel, calibrated from their scale bars and resampled to a common pixel size.
2. Deconvolution. A Richardson–Lucy deconvolution sharpens the thin tubules.
3. Segmentation and skeletonisation. A tubeness filter and an automatic threshold give the binary mask of the mitochondria, which is reduced to a skeleton one pixel wide.
4. Network. The skeleton becomes a graph, from which all metrics are computed.
5. Graph structure. A node-link drawing of the graph of each cell, with its largest network highlighted.

A sixth tab compares the metrics of the two cells in a table and a bar chart, and a last tab lists the definitions of the metrics.

**What you need.** Two single-channel or colour fluorescence images (PNG, TIFF, JPEG or BMP), each with a scale bar of known length drawn as a bright horizontal bar in its bottom-right corner. Chapter 5 explains how to prepare them.

**What you get.** Figures of every step, CSV and JSON tables of the metrics of each cell, a comparison table of the two cells and, on request, an Excel workbook (Chapter 8).

**Ways to use it.** Most users work in the graphical window described in Chapters 2 to 4. The same pipeline runs from the command line, and the Python package `mitochondrial_network_analysis` can be used in scripts (Chapter 9).

# 2 Installation

Car.Ma._Cell is a Python program. You do not need to install Python by hand: for Windows and macOS the folder contains two files, one that installs everything once and one that starts the program.

| Operating system | 1) Install once | 2) Start the program |
|---|---|---|
| Windows 10 or 11 | `Install_Windows.bat` | `Avvia_Windows.bat` |
| macOS 11 (Big Sur) or newer | `Install_Mac.command` | `Avvia_Mac.command` |

## 2.1 Getting the program

Download the source code of the latest release from `https://github.com/colivieri89/Car.Ma._Cell/releases/latest` (the archive `Source code (zip)` under *Assets*), or use **Code** → **Download ZIP** on the repository page. Unzip it into a folder you keep, for example in *Documents*. With git you can also clone the repository:

```
git clone https://github.com/colivieri89/Car.Ma._Cell.git
```

## 2.2 Windows

1. Double-click `Install_Windows.bat`. The installer looks for Python 3.9 or newer. If there is none, it installs Python 3.12 for the current user only, without an administrator password, with `winget` when available and otherwise with the official installer from `https://www.python.org`.
2. It creates a private environment in the folder `.venv` next to the installer and installs the scientific libraries listed in `requirements.txt` (NumPy, SciPy, scikit-image, pandas, NetworkX, Matplotlib, tifffile, imageio, Pillow, openpyxl).
3. When the window says *Setup complete*, close it. From now on, double-click `Avvia_Windows.bat` to start the program.

If Windows shows *Windows protected your PC* (SmartScreen) the first time, click **More info** and then **Run anyway**: the batch files are not signed.

## 2.3 macOS

1. Double-click `Install_Mac.command`. The installer looks for a Python 3.9 or newer that has Tkinter, the toolkit of the program window. If there is none, it downloads Python 3.12 from `https://www.python.org` and asks once for your password to install it.
2. It creates the environment `.venv` in the program folder and installs the libraries.
3. When the window says *Setup complete*, press Return to close it. From now on, double-click `Avvia_Mac.command` to start the program.

Intel and Apple silicon Macs are both supported.

**Gatekeeper**

The first time, macOS may refuse to open a `.command` file *because it is from an unidentified developer*. Right-click (or Control-click) the file, choose **Open** and confirm with **Open**; macOS remembers the choice. To clear the block for the whole folder at once, open Terminal and run

```
xattr -dr com.apple.quarantine "/path/to/Car.Ma._Cell"
```

If the Finder opens the file as text instead of running it, make the two files executable from Terminal, inside the program folder:

```
chmod +x Install_Mac.command Avvia_Mac.command
```

## 2.4 Linux and other systems

Install Python 3.9 or newer with the Tk bindings (on Debian and Ubuntu: `sudo apt install python3 python3-venv python3-tk python3-pip`), then, from the program folder:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/python gui.py
```

The same commands work on Windows and macOS for users who prefer a terminal (on Windows the interpreter is `.venv\Scripts\python.exe`).

## 2.5 Updating and removing

To update, download the new release into a new folder and run its installer once. Running the installer again in an existing folder only checks the libraries. Nothing is installed system-wide by Car.Ma._Cell itself, apart from Python when it was missing: to remove the program, delete its folder.

# 3 Quick start

The folder `examples` contains a synthetic pair of images that the program loads at start-up (Figure 3.1): `synthetic_fused.png`, a network of long branched tubules, and `synthetic_fragmented.png`, many short isolated fragments. Each carries a scale bar of 60 pixels that represents 6 µm. The images are drawn by `examples/make_synthetic_pair.py` and serve only to try the program.

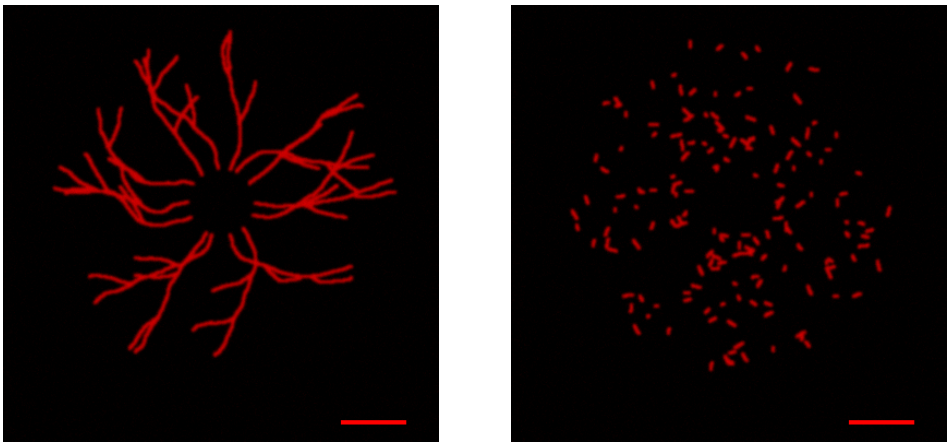


**Figure 3.1.** The synthetic example pair: fused network (left, cell A) and fragmented mitochondria (right, cell B). The bar in the bottom-right corner is the 6 µm scale bar.

1. Start the program (`Avvia_Windows.bat` or `Avvia_Mac.command`). The window opens on the *Welcome* tab (Figure 3.2).
2. Open the tab *Step 1 – Inputs*. The two example images are already chosen, with labels *Fusion* and *Fission* and scale bars of 6 µm.
3. Press **Run analysis**. The status bar at the bottom of the window reads *Running paired analysis*; the example takes a few seconds, a pair of large micrographs up to a few minutes.
4. When the status bar reads *Done*, look through the tabs from *Step 2* to *Step 6*. The fused cell has fewer components, longer branches, a connectivity index of 1 and a much higher network complexity than the fragmented one (Table 7.1).
5. Press **Save data…** to write all the metrics to an Excel workbook, and **Save pictures…** to copy all the figures to a folder of your choice.

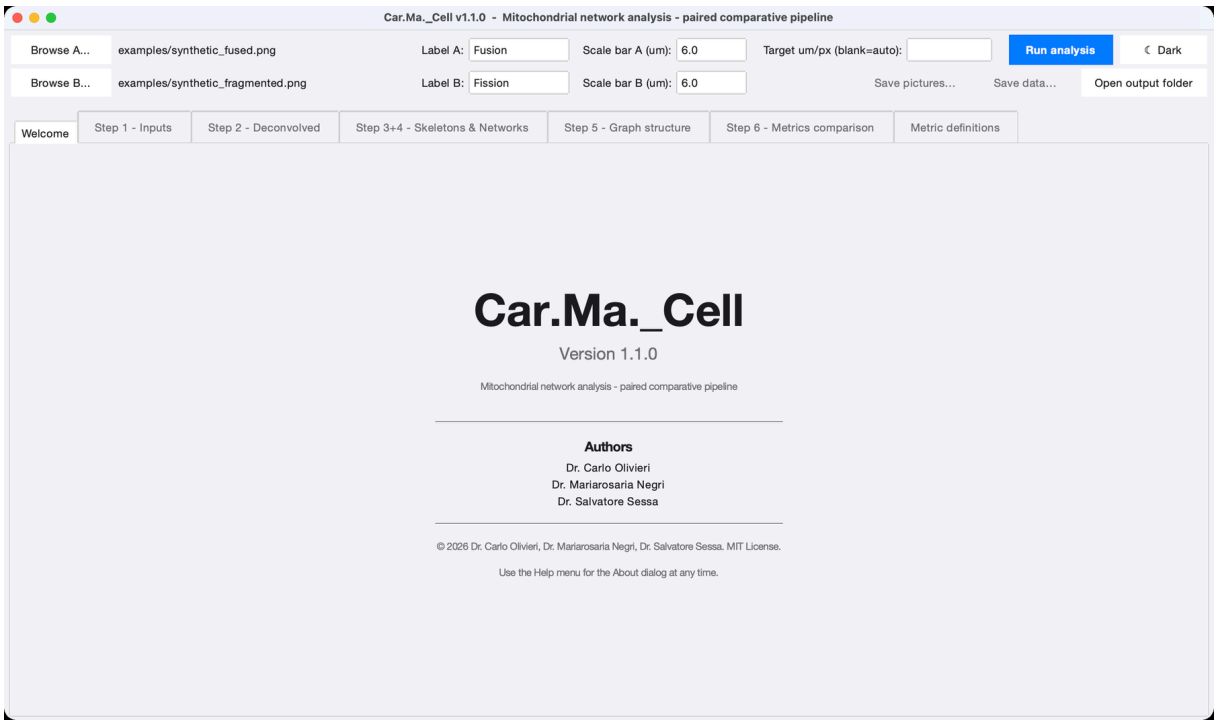



**Figure 3.2.** The *Welcome* tab, with the program version, the authors and the licence.

To analyse your own cells, press **Browse A…** and **Browse B…**, choose the two images, type the length of each scale bar in micrometres, and press **Run analysis**.

# 4 The program window

## 4.1 Toolbar

The two rows at the top of the window hold the inputs and the commands (Figure 4.1).

**Browse A… / Browse B…** Choose the image of cell A or cell B. The path appears next to the button and the image is shown in the *Step 1* tab.

**Label A / Label B** Names of the two cells, used in titles, tables, file names and the Excel workbook. The defaults are *Fusion* and *Fission*; any text works, for example *WT* and *KO*.

**Scale bar A / B (um)** Length, in micrometres, of the scale bar drawn in each image. The program measures the bar in pixels and derives the pixel size (Section 6.1).

**Target um/px** Pixel size of the common grid on which both cells are analysed. Leave it blank to use the finer of the two images; enter a value to impose another one.

**Run analysis** Applies the settings of the Step 1 and Step 2 tabs and runs the pipeline on both images. The button is disabled while the analysis runs.

**Dark / Light** Switches between the light and the dark appearance (also in the *View* menu).

**Save pictures…** Copies every PNG figure of the output folder, with its sub-folders, to a folder of your choice. Available after a run.

**Save data…** Writes the global metrics of both cells to one Excel workbook (Section 8.1). Available after a run.

**Open output folder** Opens the folder `main_output` of the program in the file manager.

The *Help* menu opens the *About* dialog with version, authors and licence. The status bar at the bottom shows the progress of the analysis and, at the end, the common pixel size, the length of each scale bar in pixels and the output folder.

## 4.2 Tabs

**Welcome** Name, version, authors and licence (Figure 3.2).

**Step 1 – Inputs** The two input images and the settings of Step 2 (Figure 4.1). Here you can also exclude areas of the images (Section 4.4).

**Step 2 – Deconvolved** The deconvolved images, at the common pixel size, and the settings of Step 3 (Figure 4.2).

**Step 3+4 – Skeletons & Networks** For each cell, the skeleton and the network, with every branch in its own colour, junctions in red and endpoints in cyan (Figure 4.3).

**Step 5 – Graph structure** A node-link drawing of the graph of each cell at the same scale: grey lines are the mitochondrial segments, orange dots the junctions, blue dots the endpoints, and the largest connected network is drawn in green (Figure 4.4).

**Step 6 – Metrics comparison** The table of the global metrics of the two cells, by group, and the bar chart of ten key metrics (Figure 4.5). After the first run, the drop-down *Bar-chart colours* switches the chart between `color` (the colour-blind safe blue and vermillion of the Okabe–Ito palette) and `mono` (grey and black) without running the analysis again.

**Metric definitions** The formula and meaning of each metric of the table (Figure 4.6). Chapter 7 gives the complete list.

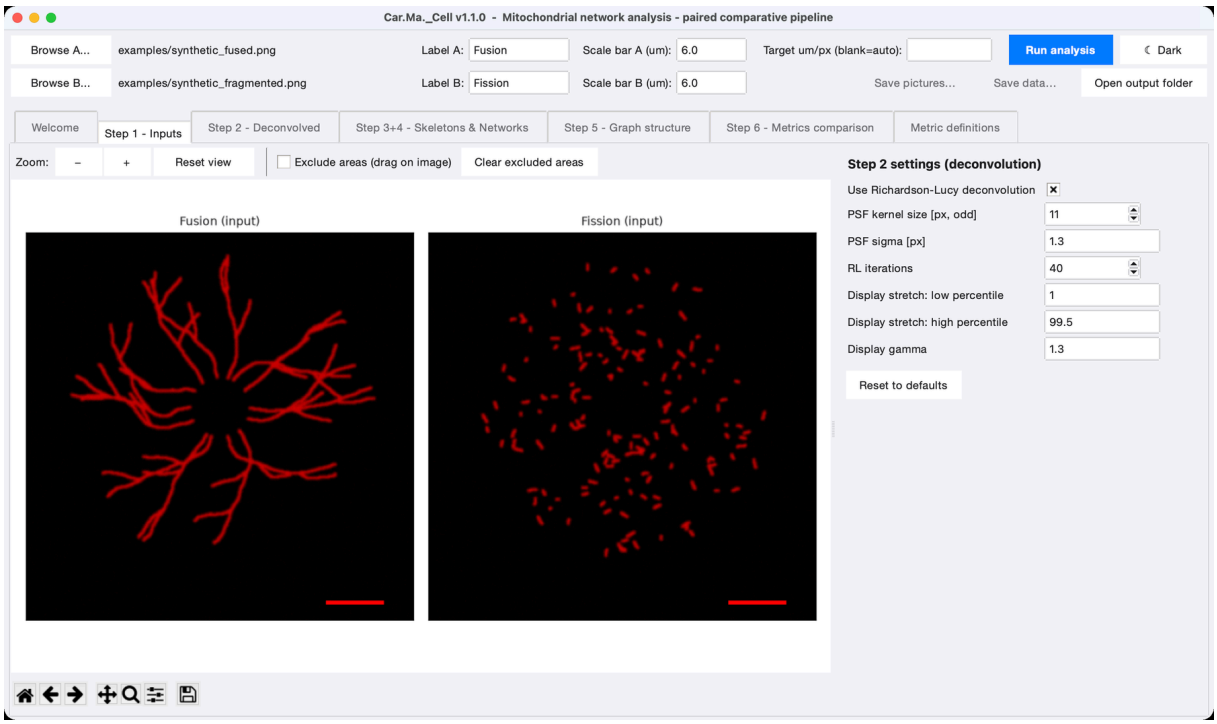



**Figure 4.1.** The *Step 1 – Inputs* tab with the example pair. On the right, the settings of the deconvolution (Step 2).

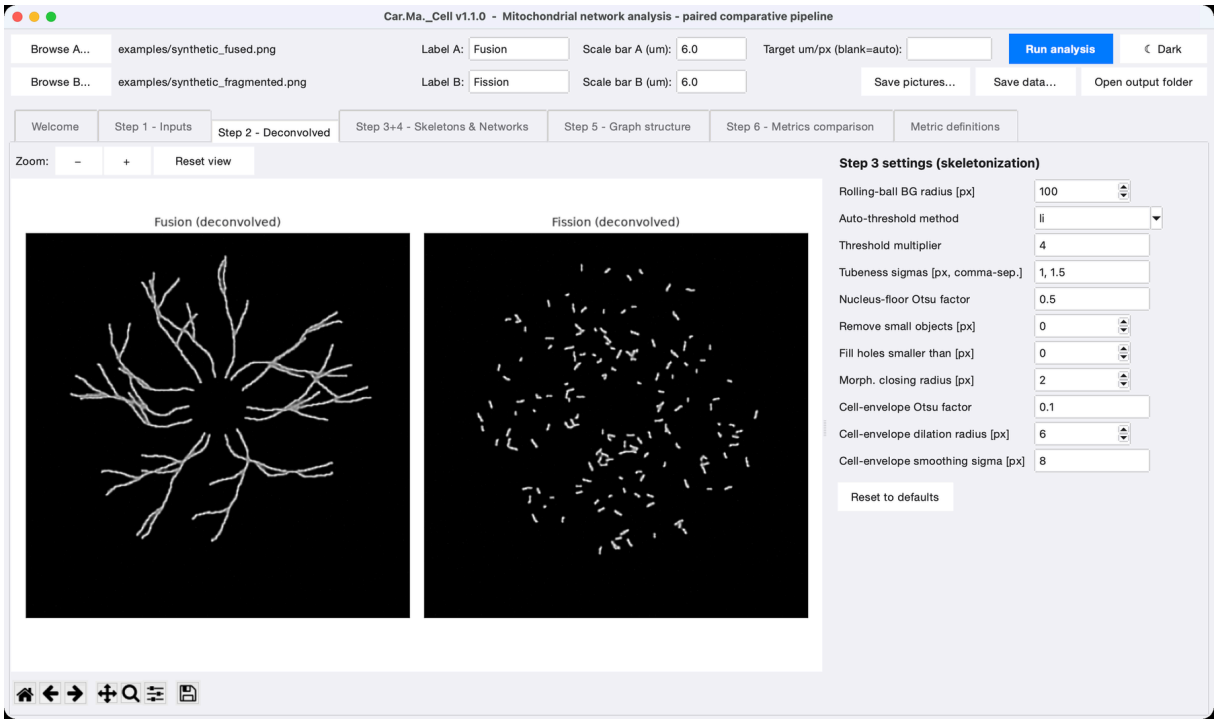



**Figure 4.2.** The *Step 2 – Deconvolved* tab after a run. On the right, the settings of segmentation and skeletonisation (Step 3).

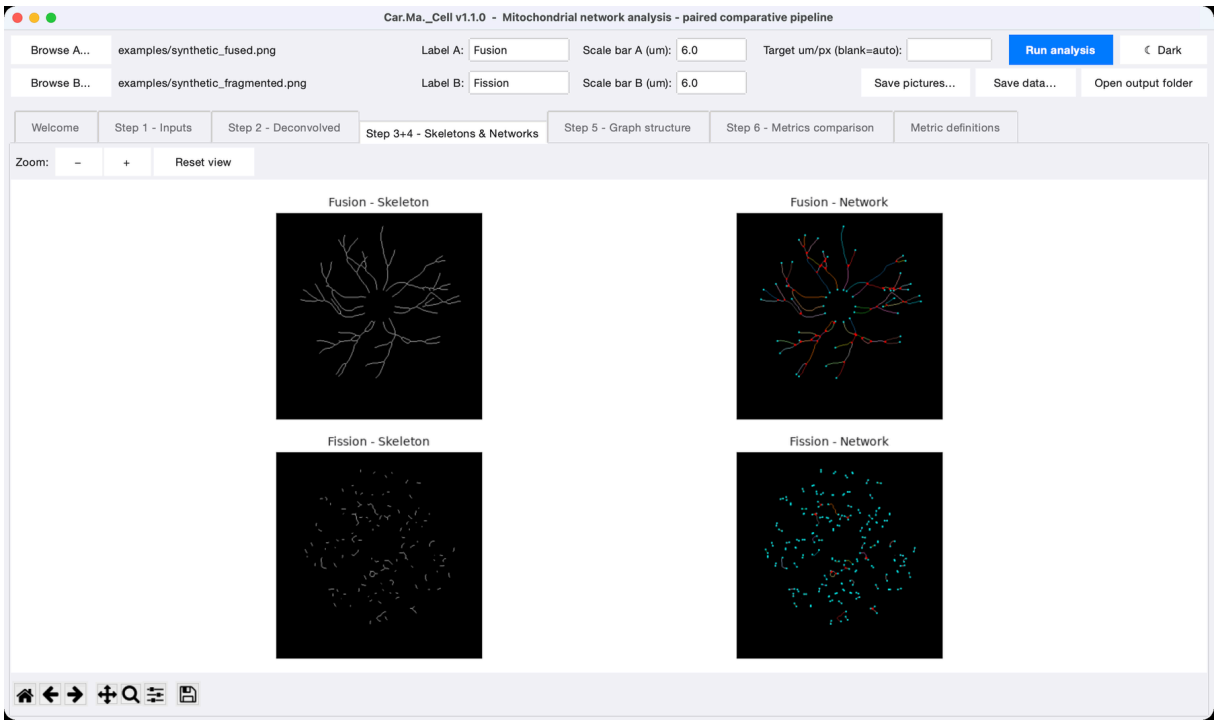
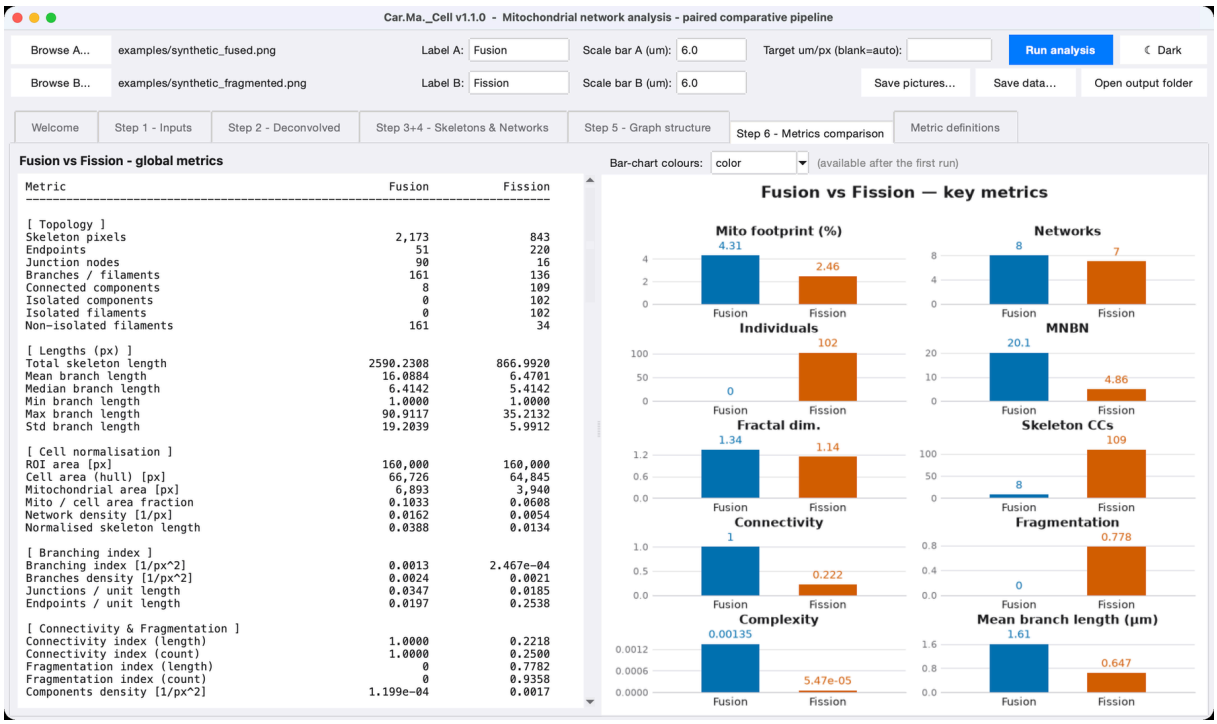



**Figure 4.3.** The *Step 3+4* tab: skeletons (left) and networks (right) of the two cells.




**Figure 4.4.** The *Step 5 – Graph structure* tab. The largest network of each cell is drawn in green.




| Metric | Fusion | Fission |
|---|---|---|
| [ Topology ] | | |
| Skeleton pixels | 2,173 | 843 |
| Endpoints | 51 | 220 |
| Junction nodes | 90 | 16 |
| Branches / filaments | 161 | 136 |
| Connected components | 8 | 109 |
| Isolated components | 0 | 102 |
| Isolated filaments | 0 | 102 |
| Non-isolated filaments | 161 | 34 |
| [ Lengths (px) ] | | |
| Total skeleton length | 2590.2308 | 866.9920 |
| Mean branch length | 16.0884 | 6.4701 |
| Median branch length | 6.4142 | 5.4142 |
| Min branch length | 1.0000 | 1.0000 |
| Max branch length | 90.9117 | 35.2132 |
| Std branch length | 19.2039 | 5.9912 |
| [ Cell normalisation ] | | |
| ROI area [px] | 160,000 | 160,000 |
| Cell area (hull) [px] | 66,726 | 64,845 |
| Mitochondrial area [px] | 6,893 | 3,940 |
| Mito / cell area fraction | 0.1033 | 0.0608 |
| Network density [1/px] | 0.0162 | 0.0054 |
| Normalised skeleton length | 0.0388 | 0.0134 |
| [ Branching index ] | | |
| Branching index [1/px^2] | 0.0013 | 2.467e-04 |
| Branches density [1/px^2] | 0.0024 | 0.0021 |
| Junctions / unit length | 0.0347 | 0.0185 |
| Endpoints / unit length | 0.0197 | 0.2538 |
| [ Connectivity & Fragmentation ] | | |
| Connectivity index (length) | 1.0000 | 0.2218 |
| Connectivity index (count) | 1.0000 | 0.2500 |
| Fragmentation index (length) | 0 | 0.7782 |
| Fragmentation index (count) | 0 | 0.9358 |
| Components density [1/px^2] | 1.199e-04 | 0.0017 |

**Figure 4.5.** The *Step 6 – Metrics comparison* tab: table of the global metrics and bar chart of the key metrics.

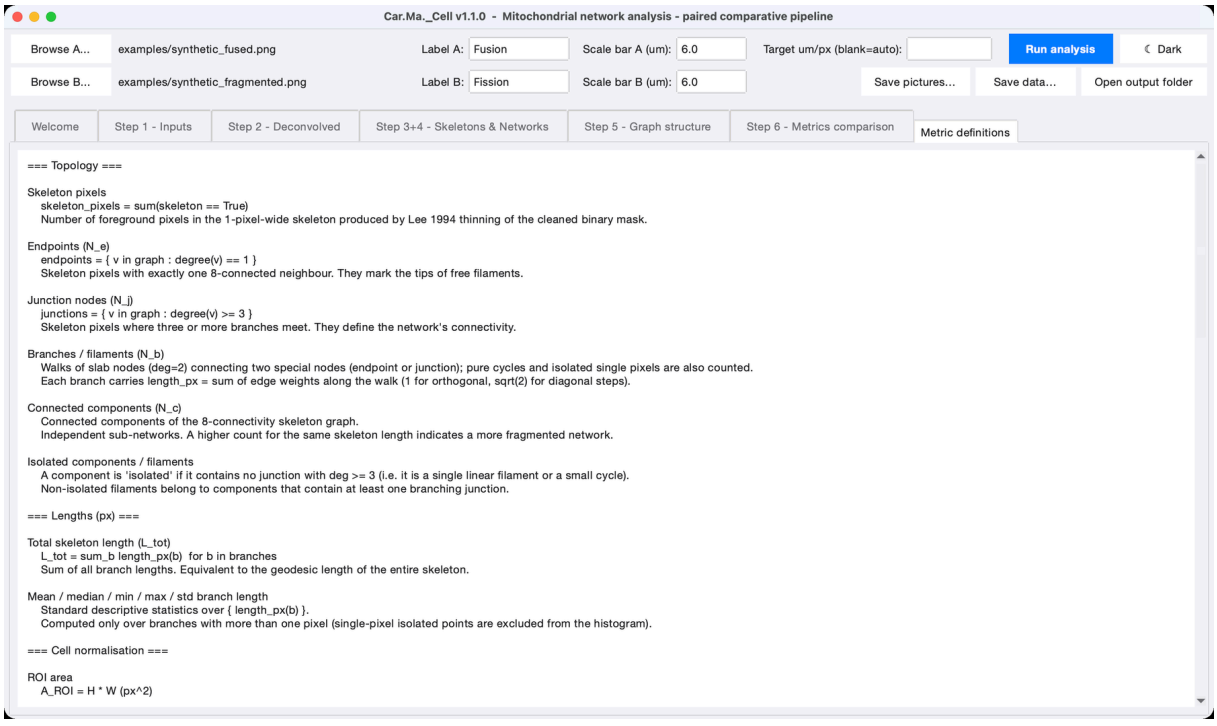



**Figure 4.6.** The *Metric definitions* tab.

## 4.3 Zoom and pan

Above the images of each tab, **–** and **+** zoom out and in on all panels together, and **Reset view** restores the full view. The bar below the images is the standard Matplotlib toolbar: the cross arrows pan, the magnifier zooms on a rectangle, the house restores the view, and the disk saves the panel as an image.

## 4.4 Excluding areas

Debris, a neighbouring cell or a label in the field can be removed from the analysis. In the *Step 1* tab, tick **Exclude areas (drag on image)** and drag a rectangle on either image; repeat for as many areas as needed. The excluded rectangles are drawn on the images and are set to background before the analysis of that cell. **Clear excluded areas** removes them all. The rectangles are stored as fractions of the width and height of each image, so they follow the image when it is resampled.

## 4.5 Settings

The panels on the right of the Step 1 and Step 2 tabs show the main parameters with their default values (Tables 6.1 and 6.2). Edited values are applied when you press **Run analysis** and stay in force until the window is closed; **Reset to defaults** restores the values written in `main.py`. Section 6.2 explains what each parameter does.

## 4.6 Appearance

**Dark** switches the whole window, figures included, to a dark appearance (Figure 4.7); **Light** switches it back. The figures saved to disk always have a white background.




**Figure 4.7.** The dark appearance.

# 5 Preparing the images

- One cell per image, roughly centred, with its mitochondria labelled by a fluorescent dye or protein. A field with several cells works if the others are excluded (Section 4.4).
- Formats: PNG, TIFF, JPEG or BMP, 8 or 16 bit. In a colour image the program uses the channel with the highest mean intensity and ignores an alpha channel; save the mitochondrial channel alone if the image has several stains.
- Scale bar: a solid, bright, horizontal bar in the bottom-right corner, as written by most microscope and Fiji exports. Its length in micrometres is entered in the toolbar. The bar is found automatically (Section 6.1) and is erased before the analysis, together with any bright text near the bottom edge.
- Same acquisition: the two cells should come from the same microscope, objective and settings. The program compensates different pixel sizes, not different optics or exposure.
- Avoid saturated images and strong compression (JPEG at low quality), which merge neighbouring tubules.

# 6 How the analysis works

## 6.1 Calibration and common grid

For each image the program looks for the scale bar in the bottom 20 % of the height and the right 60 % of the width. There it marks the pixels at or above the 99th percentile of the intensity of the whole image and takes the longest horizontal run of marked pixels in a single row as the bar. With a bar of $n$ pixels and a declared length $\ell$, the pixel size is $s = \ell/n$ (µm per pixel). If no bar is found, the analysis stops with the message *Could not detect a scale bar*.

Both images are then resampled with bicubic interpolation to a common pixel size: the finer of the two by default (the coarser image is enlarged), or the value of **Target um/px**. All lengths and areas of the two cells are therefore measured on the same grid. In the example both bars measure 60 pixels and the common pixel size is 0.1 µm.

## 6.2 The pipeline

The two cells go through the same steps with the same parameters.

### Step 1: input

The intensity is normalised to $[0, 1]$ and the excluded areas are set to zero. Bright annotations in the bottom 18 % of the image (the scale bar and its text) are found with the 99th percentile and erased, with a margin of 6 pixels, so that they do not enter the mask.

### Step 2: deconvolution

A Richardson–Lucy deconvolution with a Gaussian point-spread function (PSF) sharpens the tubules; the result is normalised to $[0, 1]$. Table 6.1 lists the settings. The three *display* settings change only the picture shown in the Step 2 tab and saved as `step2_deconvolved.png` (percentile stretch and gamma), not the analysis. When **Use Richardson-Lucy deconvolution** is off, Step 3 works on the normalised input.

**Table 6.1.** Settings of Step 2 (Step 1 tab), with their names in `main.py` and their default values.

| Setting | Name in `main.py` | Default |
|---|---|---|
| Use Richardson-Lucy deconvolution | `USE_DECONV` | on |
| PSF kernel size [px, odd] | `PSF_SIZE` | 11 |
| PSF sigma [px] | `PSF_SIGMA` | 1.3 |
| RL iterations | `DECONV_ITERATIONS` | 40 |
| Display stretch: low percentile | `DISPLAY_LOW_PCT` | 1 |
| Display stretch: high percentile | `DISPLAY_HIGH_PCT` | 99.5 |
| Display gamma | `DISPLAY_GAMMA` | 1.3 |

### Step 3: segmentation and skeletonisation

The binary mask of the mitochondria is the intersection of three conditions, computed on the deconvolved image:

1. Cell envelope. The image is smoothed with a Gaussian of width *cell-envelope smoothing sigma*; the pixels above *cell-envelope Otsu factor* times the Otsu threshold form a permissive mask of the cell body, dilated by a disk of *cell-envelope dilation radius*. If this mask covers less than 1 % or more than 95 % of the image, the plain Otsu threshold is used instead. Signal outside the envelope is ignored.
2. Tubeness. A rolling-ball background of the given radius is subtracted, a median filter of radius 1 pixel removes isolated noise, and the Sato tubeness filter for bright tubular structures, computed at the scales *tubeness sigmas*, enhances the tubules. The tubeness is normalised to a maximum of 1 and thresholded with the chosen automatic method (`li, otsu, triangle, yen, mean, isodata, minimum`) multiplied by the *threshold multiplier*.
3. Intensity floor. The deconvolved intensity must exceed *nucleus-floor Otsu factor* times the Otsu threshold of the intensities inside the envelope, which removes the dim perinuclear haze.

The mask is closed with a disk of *closing radius*; holes and objects smaller than the given sizes are filled and removed (0 turns these two steps off). Finally the mask is reduced to a skeleton one pixel wide with the algorithm of Lee, Kashyap and Chu (1994).

**Threshold method and multiplier**

The default multiplier, 4.0, is tuned for the Li method, whose threshold on the tubeness image is low. With Otsu, IsoData, Yen or Minimum the threshold is higher and a multiplier near 1 is needed; if the multiplier is too high, the analysis stops with *Binary mask is empty*. The Minimum method needs a histogram with two peaks; when it cannot find them, the program uses Otsu and writes a warning in the terminal.

**Table 6.2.** Settings of Step 3 (Step 2 tab), with their names in `main.py` and their default values.

| Setting | Name in `main.py` | Default |
|---|---|---|
| Rolling-ball BG radius [px] | `BG_RADIUS` | 100 |
| Auto-threshold method | `THRESHOLD_METHOD` | li |
| Threshold multiplier | `THRESHOLD_FACTOR` | 4 |
| Tubeness sigmas [px, comma-sep.] | `FRANGI_SIGMAS` | 1, 1.5 |
| Nucleus-floor Otsu factor | `NUCLEUS_FLOOR_FACTOR` | 0.5 |
| Remove small objects [px] | `MIN_OBJECT_SIZE` | 0 |
| Fill holes smaller than [px] | `MIN_HOLE_SIZE` | 0 |
| Morph. closing radius [px] | `CLOSING_RADIUS` | 2 |
| Cell-envelope Otsu factor | `CELL_MASK_OTSU_FACTOR` | 0.1 |
| Cell-envelope dilation radius [px] | `CELL_MASK_DILATION_RADIUS` | 6 |
| Cell-envelope smoothing sigma [px] | `CELL_MASK_SMOOTH_SIGMA` | 8 |

## Step 4: network graph

Every skeleton pixel becomes a node of a graph in which neighbouring pixels (8-connectivity) are joined by edges of length 1 (side) or $\sqrt{2}$ (diagonal). By its number of neighbours (degree $d$) a node is an *endpoint* ($d = 1$), a *slab* pixel ($d = 2$) or a *junction* ($d \geq 3$). A *branch* is a chain of slab pixels between two junctions or endpoints, a closed loop of slab pixels is one branch, and an isolated pixel is a branch of zero length. A connected component that contains at least one junction is a *network*; a component without junctions is an *individual mitochondrion* (a single filament or punctum).

## Step 5: graph structure

The graph of each cell is drawn as lines between junctions and endpoints, at the same physical scale for both cells, with the largest network highlighted.

## 6.3 Advanced parameters

Some parameters are not in the window and are set as constants at the top of `main.py`: the theoretical PSF (`PSF_MODE = "theoretical"`, with $\sigma = 0.21\,\lambda_{\mathrm{em}}/\mathrm{NA}$ from `PSF_NA`, `PSF_WAVELENGTH_EM_NM` and `PSF_PIXEL_SIZE_NM`), an alternative segmentation that follows the Fiji commands Subtract Background, Normalize and Auto Threshold on an 8-bit image (`FIJI_MODE = True`), the length of the scale bar drawn on the comparison figures (`DISPLAY_SCALEBAR_UM`) and the margins of the annotation removal. Edit them with a text editor, save, and start the program again.

# 7 Metrics

Notation: $L$ total skeleton length, $N_b$ branches, $N_j$ junctions ($d \geq 3$), $N_e$ endpoints ($d = 1$), $N_c$ connected components, $A_{\mathrm{ROI}}$ area of the analysed image, $A_{\mathrm{cell}}$ area of the convex hull of the mitochondrial mask (a proxy of the cell footprint, robust to cropping), $A_{\mathrm{mito}}$ area of the mask. Lengths and areas are given in pixels of the common grid and, where marked, in µm and µm$^2$. Every metric of the table of Step 6 is described in the *Metric definitions* tab as well.

## 7.1 Topology and lengths

**Skeleton pixels** number of pixels of the skeleton, equal to the number of nodes of the graph.

**Endpoints, junction nodes, branches, connected components** $N_e, N_j, N_b, N_c$.

**Isolated components and filaments** components without any junction, and their branches; *non-isolated filaments* are the branches of networks.

**Total skeleton length** $L = \sum_b \ell_b$, the sum of the branch lengths.

**Branch length statistics** mean, median, minimum, maximum and standard deviation of the branch lengths, in pixels and µm.

## 7.2 Cell normalisation

**ROI area** $A_{\mathrm{ROI}}$, width times height of the analysed image.

**Cell area (hull)** $A_{\mathrm{cell}}$.

**Mitochondrial area** $A_{\mathrm{mito}}$, foreground pixels of the cleaned mask.

**Mito / cell area fraction** $f = A_{\mathrm{mito}}/A_{\mathrm{cell}}$.

**Network density** $\rho = L/A_{\mathrm{ROI}}$, in px$^{-1}$ and µm$^{-1}$.

**Normalised skeleton length** $L/A_{\mathrm{cell}}$, in px$^{-1}$.

## 7.3 Branching

**Branching index** $\mathrm{BI} = N_j/A_{\mathrm{cell}}$, also per µm$^2$.

**Branches density** $N_b/A_{\mathrm{cell}}$.

**Junctions and endpoints per unit length** $N_j/L$ and $N_e/L$. Many endpoints per unit length signal a fragmented network.

## 7.4 Connectivity and fragmentation

**Connectivity index, length-based** $C_L = L_{\mathrm{net}}/L$, the fraction of the skeleton length that belongs to networks; 1 for a fully connected network, 0 when all mitochondria are isolated.

**Connectivity index, count-based** $C_N = N_{b,\mathrm{net}}/N_b$, the fraction of branches that belong to networks.

**Fragmentation index, length-based** $F_L = 1 - C_L$.

**Fragmentation index, count-based** $F_N = N_{c,\mathrm{iso}}/N_c$, the fraction of components without junctions.

**Components density** $N_c/A_{\mathrm{cell}}$.

**Networks and individuals** number of networks $N_{\mathrm{net}}$, number of individual mitochondria $N_{\mathrm{ind}}$, and their ratio $N_{\mathrm{net}}/N_{\mathrm{ind}}$.

**MNBN** mitochondrial network branching number, the mean number of branches per network (Valente et al., 2017), reported as `mitochondrial_connectivity_index`; the distributions of the branches per network and of the network sizes are reported too.

## 7.5 Network complexity index

$$\mathrm{NCI} \;=\; \frac{N_j}{L} \cdot \frac{L}{A_{\mathrm{cell}}} \cdot C_L \;=\; \frac{N_j\, C_L}{A_{\mathrm{cell}}},$$

in $\mathrm{px}^{-2}$ of the common grid. It combines the intensity of branching, the skeleton length per unit cell area and the connectivity: it is high for networks that are dense, branched and connected at the same time, and low for sparse or fragmented ones. Since both cells share the same grid, the values of a pair are directly comparable.

## 7.6 Object shape and fractal dimension

Each connected object of the mask (a labelled region) gives its area, its aspect ratio $\ell_{\mathrm{major}}/\ell_{\mathrm{minor}}$ and its circularity $4\pi A/P^2$, with $A$ and $P$ its area and perimeter. The program reports the statistics of these three distributions, the number of objects, the total mitochondrial area and the footprint, the mitochondrial area as a percentage of the image. The fractal dimension of the mask is the slope of $\log N(s)$ against $\log(1/s)$, with $N(s)$ the number of boxes of side $s = 2, 4, 8, 16, 32, 64$ pixels that contain part of the mask. Derived indices: branching per object ($N_j$ over the number of objects), branching per $\mu\mathrm{m}^2$ of mitochondrial area, and fragmentation by objects (individual mitochondria over the number of objects).

## 7.7 Node-degree distribution

For each degree $K$ up to the largest one found, the number $N_K$ of skeleton pixels with $K$ neighbours and its percentage of all nodes, plus the total of the nodes with $K \geq 3$.

## 7.8 Distribution statistics

For every distribution (branch lengths, branches per network, network sizes, object areas, aspect ratios, circularities) the program gives: number $n$, mean, standard deviation, median, minimum, maximum, coefficient of variation $100\,\sigma/\mu$, skewness, excess kurtosis, 25th and 75th percentiles and interquartile range. The keys end in `_n`, `_mean`, `_sd`, `_median`, `_min`, `_max`, `_cv_pct`, `_skewness`, `_kurtosis`, `_p25`, `_p75`, `_iqr`.

## 7.9 The bar chart

The bar chart of Step 6 and `comparison_metrics_bars.png` show ten key metrics: mitochondrial footprint (%), networks, individuals, MNBN, fractal dimension, skeleton components, connectivity $C_L$, fragmentation $F_L$, network complexity and mean branch length (µm). Each bar carries its value.

## 7.10 Example

Table 7.1 lists some metrics of the example pair, as computed by this version of the program, and Figure 7.1 shows the bar chart saved by the program.

**Table 7.1.** Key metrics of the synthetic example pair.

| Metric | Fusion (fused) | Fission (fragmented) |
|---|---|---|
| Junction nodes $N_j$ | 90 | 16 |
| Endpoints $N_e$ | 51 | 220 |
| Branches $N_b$ | 161 | 136 |
| Connected components $N_c$ | 8 | 109 |
| Networks | 8 | 7 |
| Individual mitochondria | 0 | 102 |
| Total skeleton length (µm) | 259 | 86.7 |
| Mean branch length (µm) | 1.609 | 0.647 |
| MNBN (branches per network) | 20.12 | 4.857 |
| Connectivity index $C_L$ | 1 | 0.2218 |
| Fragmentation index $F_L$ | 0 | 0.7782 |
| Network complexity index (px$^{-2}$) | $1.35 \times 10^{-3}$ | $5.47 \times 10^{-5}$ |
| Fractal dimension | 1.337 | 1.138 |

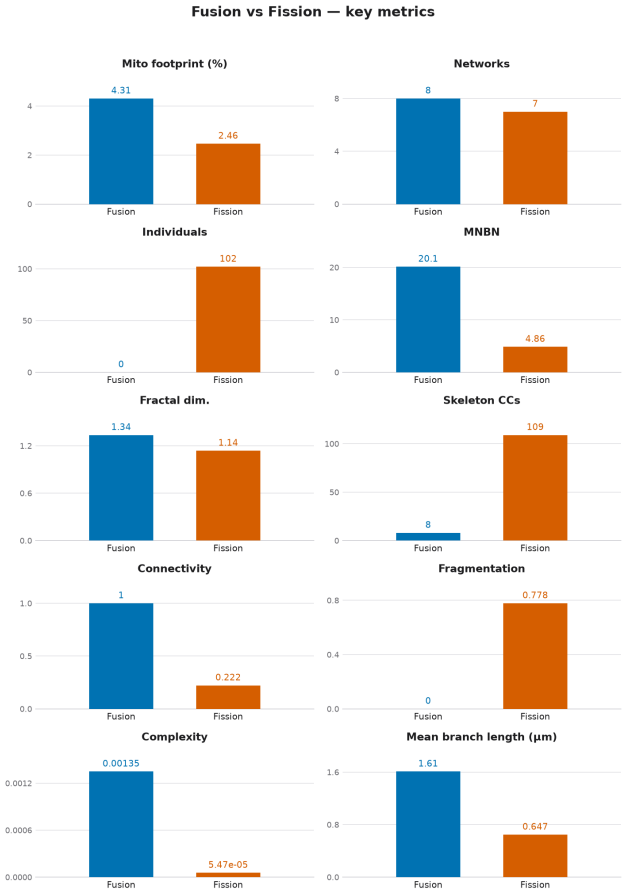



**Figure 7.1.** `comparison_metrics_bars.png` of the example pair.

# 8 Output files

Every run writes its results into the folder `main_output` of the program, with one sub-folder per cell named after its label (in lower case, with spaces and symbols replaced by underscores). A new run with the same labels overwrites the previous files: copy the folder, or use **Save pictures…** and **Save data…**, before analysing the next pair.

| File | Content |
|---|---|
| *In* `main_output` | |
| `comparison_metrics.csv` | every global metric of both cells, one row per metric: columns `metric`, label A, label B, `delta_A_minus_B` and `ratio_A_over_B` |
| `comparison_metrics_bars.png` | bar chart of the key metrics |
| `comparison_step2.png` … `comparison_step5.png` | the two cells side by side at each step, at the same scale, with a scale bar |
| *In each cell folder* | |
| `step2_deconvolved.png` | deconvolved image (display stretch) |
| `step3_skeleton.png` | skeleton |
| `step4_network.png` | network: branches, junctions and endpoints |
| `step5_graph.png` | graph structure |
| `step4_global_metrics.json`, `.csv` | all global metrics of the cell (Chapter 7) |
| `step4_branches.csv` | one row per branch: component, start and end pixel and type, pixels, length |
| `step4_branch_metrics.csv` | as above, with length in µm and whether the branch is an isolated filament |
| `step4_component_metrics.csv` | one row per connected component: length, endpoints, junctions, branches, bounding box, centroid |
| `step4_junctions.csv` | position and degree of each junction pixel |
| `step4_analyze_skeleton.csv` | branch table in the layout of the Fiji *Analyze Skeleton* plug-in |

Pixel coordinates are given as row and column of the common grid, from the top-left corner.

## 8.1 Excel workbook

**Save data…** writes one sheet, *Metrics*, with the name of each metric in the first column and its value for each cell in the following columns, headed by the labels. The metrics shown in Step 6 come first, in the same order, followed by all the others. The default file name is `CarMaCell_metrics.xlsx`.

## 8.2 Example results

The folder `examples/example_results` contains the output of a paired analysis of two fluorescence micrographs, with labels *Fusion* and *Fission*, as an example of what a run on real cells produces.

# 9 Command line and Python

## 9.1 Paired analysis from the command line

From the program folder, with the interpreter of `.venv`:

```
python main.py <image A> <scale bar A in um> <image B> <scale bar B in um>
python main.py examples/synthetic_fused.png 6 examples/synthetic_fragmented.png 6
```

The run uses the labels *Fusion* and *Fission*, the defaults of `main.py`, and writes into `main_output`. Without arguments, `python main.py` opens the window.

## 9.2 In a Python script

```
import main
result = main.process_pair("cellA.tif", 10.0, "cellB.tif", 18.0,
                           output_dir="results", label_a="WT", label_b="KO",
                           verbose=False)
metrics_a = result["fusion"]["global_metrics"]   # dict of all metrics of cell A
table = result["comparison_df"]                  # pandas DataFrame of the comparison
```

The keys `"fusion"` and `"fission"` hold the results of cell A and cell B whatever their labels. Settings are module constants and can be changed before the call, for example `main.THRESHOLD_METHOD = "otsu"`; `main.THRESHOLD_FACTOR = 1.0`. The arguments `exclude_regions_a` and `exclude_regions_b` take lists of rectangles `(x0, y0, x1, y1)` in fractions of the image width and height.

## 9.3 Single images: the package

The package `mitochondrial_network_analysis` holds the building blocks of the pipeline (input, deconvolution, segmentation, skeleton, graph, metrics and figures) and a separate single-image pipeline with its own command line:

```
python -m mitochondrial_network_analysis.cli --input cell.tif --output results \
       --pixel-size-um 0.1
python -m mitochondrial_network_analysis.cli --help
```

Its defaults (Otsu threshold, no tubeness filter, 30 deconvolution iterations) differ from those of the paired analysis, so its results are not directly comparable with those of the window. `examples/run_example.py` runs it on a synthetic image; installing the folder with `pip install .` also provides the command `mito-analyze`.

## 9.4 Tests

The folder `tests` checks the calibration, the outputs and the metrics of the paired analysis on the synthetic pair, and the agreement of the version numbers. Run them with `python -m pytest` after `pip install pytest`. They run on every change of the repository on Windows, macOS and Linux.

# 10 Troubleshooting

***Could not detect a scale bar*** The bar must be a bright horizontal line in the bottom-right corner, among the brightest pixels of the image. Re-export the image with its scale bar burnt in, or add one in Fiji (*Analyze* → *Tools* → *Scale Bar*, solid white, bottom right).

***Binary mask is empty*** The threshold removed all signal: lower the *threshold multiplier* (for methods other than Li, start from 1) or check that the image contains the mitochondrial channel.

***Skeleton is empty*** The cleaning removed everything: set *remove small objects* and *fill holes* to 0 and check the mask in the Step 3+4 tab.

**Too many small fragments** Raise the threshold multiplier or *remove small objects*, or exclude areas with debris.

**Tubules broken in pieces** Lower the threshold multiplier, increase the closing radius, or turn the deconvolution on if it was off.

***… is not installed yet*** The start file was used before the installer. Run `Install_Windows.bat` or `Install_Mac.command` first.

**The installer cannot download** Check the internet connection; the installer needs `https://www.python.org` (only if Python is missing) and the Python Package Index.

**The program does not start after moving the folder** The environment `.venv` belongs to the folder where it was created. Delete `.venv` and run the installer again.

**Messages and errors** Problems are shown in a dialog and in the terminal window that opens with the program; quote that text when you report a problem at `https://github.com/colivieri89/Car.Ma._Cell/issues`.

# 11 Citation, licence and authors

**Citation.** If you use Car.Ma._Cell in your work, please cite it as described in the file `CITATION.cff` of the repository. Every release is archived on Zenodo, which assigns it a DOI; the DOI shown in the README refers to all versions.

**Licence.** Car.Ma._Cell is released under the MIT License (file `LICENSE`). It uses NumPy, SciPy, scikit-image, pandas, NetworkX, Matplotlib, tifffile, imageio, Pillow and openpyxl, each under its own licence.

**Authors.** Carlo Olivieri, Mariarosaria Negri, Salvatore Sessa.

**References.**


- T.-C. Lee, R. L. Kashyap and C.-N. Chu, Building skeleton models via 3-D medial surface/axis thinning algorithms, *CVGIP: Graphical Models and Image Processing* 56 (1994) 462–478.
- Y. Sato et al., Three-dimensional multi-scale line filter for segmentation and visualization of curvilinear structures in medical images, *Medical Image Analysis* 2 (1998) 143–168.
- W. H. Richardson, Bayesian-based iterative method of image restoration, *Journal of the Optical Society of America* 62 (1972) 55–59; L. B. Lucy, An iterative technique for the rectification of observed distributions, *The Astronomical Journal* 79 (1974) 745–754.
- C. H. Li and C. K. Lee, Minimum cross entropy thresholding, *Pattern Recognition* 26 (1993) 617–625.
- A. J. Valente, L. A. Maddalena, E. L. Robb, F. Moradi and J. A. Stuart, A simple ImageJ macro tool for analyzing mitochondrial network morphology in mammalian cell culture, *Acta Histochemica* 119 (2017) 315–326.
- I. Arganda-Carreras, R. Fernández-González, A. Muñoz-Barrutia and C. Ortiz-De-Solórzano, 3D reconstruction of histological sections: application to mammary gland tissue, *Microscopy Research and Technique* 73 (2010) 1019–1029 (*Analyze Skeleton*).
- B. Wong, Color blindness, *Nature Methods* 8 (2011) 441.